

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

INTELLIGENT NOSQL DATABASES

PROJECT REPORT

FRESHKART: SUBSCRIPTION-BASED E-COMMERCE PLATFORM

Submitted by

Aaruhi

Registration No.: 12311428

Programme & Section: B.Tech Computer Science and Engineering, K4E2338

Course Code: CSE494

Under the Guidance of

Dr.Manpreet Singh

U.Id: 32354

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

Submission Date: 29 October 2026

CERTIFICATE

This is to certify that the project report entitled “FreshKart: Subscription-Based E-Commerce Platform” submitted by Aaruhi ,12311428, a B.Tech Computer Science and Engineering student of the School of Computer Science and Engineering, Lovely Professional University, Phagwara, Punjab, in partial fulfilment of the requirements of CSE494 - Intelligent NoSQL Databases, is a record of the project work carried out by the student under my guidance and supervision.

The report presents an analysis of FreshKart, a subscription-based e-commerce platform built with React, Node.js, Express, MongoDB and Mongoose. It describes the application architecture, data models, shopping and subscription workflows, and findings from review of the supplied implementation.

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab

Supervisor: Dr. Manpreet Singh Sehgal

Date: 29 October 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty supervisor, Dr. Manpreet Singh Sehgal, for the guidance, feedback and encouragement provided during this project. The study of intelligent NoSQL databases helped shape the database analysis and the understanding of document modelling used in this report.

I thank the School of Computer Science and Engineering, Lovely Professional University, for providing an academic environment in which to study and develop this work. I am also grateful to the MongoDB, Mongoose, React, Node.js and Express communities, whose documentation supported the project analysis and implementation.

Finally, I thank my family, friends and classmates for their encouragement and for sharing suggestions that helped improve the project.

Abstract

FreshKart is a web application for shopping everyday groceries and household essentials through one-time purchases and recurring delivery plans. This report analyzes the supplied implementation, covering its user experience, API, MongoDB document model, order lifecycle, and subscription workflows. The codebase shows a React frontend, an Express API, and MongoDB access through Mongoose. Its main domain records are users, products, orders, and subscriptions, with embedded addresses, reviews, order lines, and tracking events.

The implementation includes account registration and login, OTP verification, product catalog management, checkout, order tracking, product reviews, and daily, weekly, or monthly subscription settings. A seeded sample catalog is available. The review also identifies practical gaps: subscription order generation is exposed without route-level protection, generation uses an incomplete shipping address for the required order schema, recurring plans do not enforce end dates in the shown generation logic, and inventory/order writes are not transactional. These are findings from static code inspection, not measured runtime test results.

Table of Contents

1. Introduction 6
1.1 Project Overview 6
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Scope 6
1.5 Method 6
2. Problem Domain and Requirements 6
2.1 Domain Overview 6
2.2 Existing Workflow 6
2.3 Functional Requirements Evident in the Code 6
2.4 Non-Functional Needs 7
3. Technology Stack and System Architecture 7
3.1 Technology Environment 7
3.2 Architecture 7
3.3 Frontend Areas 7
4. Data Source and Database Design 7
4.1 Data Source 7
4.2 Collections and Main Fields 7
4.3 Embedding and Referencing 8
4.4 Validation and Indexes 8
5. Application Implementation 8
5.1 Authentication and User Management 8
5.2 Product Catalog 8
5.3 Checkout and Orders 8
5.4 Subscription Lifecycle 8
5.5 Administration 8
6. MongoDB Operations and Integration 8
6.1 Mongoose Integration 8
6.2 CRUD Mapping 9
6.3 Document Design Considerations 9
6.4 Queries and Aggregation 9
7. Results and Implementation Analysis 9
7.1 Implemented Capabilities 9
7.2 Important Findings 9
7.3 Verification Status 10
8. Discussion and Recommendations 10
8.1 Strengths 10
8.2 Priority Improvements 10
8.3 Security and Operations 10
9. Conclusion and Future Scope 10
9.1 Conclusion 10
9.2 Future Scope 10
References 11
List of Tables 5
List of Figures and Diagrams 5
List of Domain Entities 5

List of Tables

Table 1: Functional requirements implemented in FreshKart 6
Table 2: FreshKart technology environment 7
Table 3: FreshKart MongoDB document model 7
Table 4: CRUD operations in the application 9
Table 5: Important implementation findings 9
Table 6: Priority improvements 10

List of Figures and Diagrams

Figure 1: FreshKart system architecture 7

List of Domain Entities

User, Product, Order, Subscription 5

1. Introduction

1.1 Project Overview

FreshKart is designed to make routine shopping for groceries, produce, dairy, personal care, and household goods easier. Customers can browse a catalog, maintain a cart, place an order, follow its status, and create recurring product plans. Store administrators have interfaces for managing products, customers, and orders.

1.2 Problem Statement

Customers repeatedly purchase many of the same essential items. A conventional one-off shopping flow requires them to rebuild each basket and remember delivery timing. FreshKart addresses this need by combining standard checkout with subscription frequency and status management in one storefront.

1.3 Objectives

- Provide a responsive catalog and product detail experience for everyday items.
- Support customer accounts, profile data, and OTP-based email verification.
- Handle cart checkout, order history, order status, and tracking history.
- Represent recurring deliveries with frequency, billing type, next delivery date, and lifecycle status.
- Provide administrators with protected catalog and order management screens.
- Use MongoDB documents and Mongoose models to persist application data.

1.4 Scope

The reviewed repository contains a browser application and a Node.js API backed by MongoDB. The documented implementation covers account, catalog, order, subscription, review, and administration workflows. External payment processing, production scheduling infrastructure, cloud deployment, and operational service-level evidence are not established by the supplied files and are treated as future work.

1.5 Method

The analysis is based on the project README, package manifests, route definitions, Mongoose models, middleware, and relevant controllers. It describes behavior visible in those files. It does not claim that the application was executed against a live database, nor does it report performance measurements.

2. Problem Domain and Requirements

2.1 Domain Overview

The application sits in the online grocery domain, where product availability, address accuracy, inventory, delivery status, and recurring schedules are central. A subscription is a customer-owned plan attached to a product and has a quantity, frequency, start date, next delivery date, billing type, and status.

2.2 Existing Workflow

A single-order storefront typically handles a customer's immediate basket but leaves repeat purchases to a later manual checkout. FreshKart adds recurring plan setup and a customer screen for viewing subscriptions, while preserving regular checkout for one-time purchases.

2.3 Functional Requirements Evident in the Code

Table 1: Functional requirements implemented in FreshKart

AREA	IMPLEMENTED CAPABILITY

Accounts	Registration, login, profile updates, JWT-protected endpoints, admin role checks, OTP send and verify routes.
Catalog	List and view products; admin create, update, delete; image upload; reviews and rating summary.
Checkout	Validate available stock, calculate and persist submitted totals, save shipping/payment details, create order.
Orders	Customer order history and details; admin order lists, status updates, delivery marking, tracking history.
Subscriptions	Create and list plans; customer/admin plan updates; daily, weekly, and monthly frequency values.
Administration	Dashboard routes, product and order screens, user list, protected route components.

2.4 Non-Functional Needs

- Security: protect private resources, restrict administrator actions, hash passwords, and protect OTPs.
- Reliability: keep inventory and order updates consistent when concurrent checkouts or retries occur.
- Usability: make checkout, recurring plan controls, and order progress understandable on mobile and desktop.
- Maintainability: validate inputs, keep API error responses consistent, and centralize configuration.
- Scalability: add pagination, indexes, background scheduling, and monitoring as records grow.

3. Technology Stack and System Architecture

3.1 Technology Environment

Table 2: FreshKart technology environment

LAYER	TECHNOLOGY / LIBRARY	ROLE
Frontend	React 18, React Router 6	Pages, navigation, application state.
Styling	Tailwind CSS 3, CSS	Responsive layouts and interface styling.
Backend	Node.js, Express 5	HTTP API, routing, middleware and business logic.
Database	MongoDB through Mongoose 9	Document persistence and schema mapping.
Authentication	JSON Web Token, bcryptjs	Session token and password hashing.
Other backend packages	Multer, Nodemailer, Axios	Image uploads, email delivery, HTTP requests.

3.2 Architecture

The browser renders React pages and sends API requests through the shared Axios client. Express mounts route modules under /api/users, /api/products, /api/subscriptions, /api/orders, and /api/utils. Controllers apply application rules and use Mongoose models to read or write MongoDB. Authentication middleware verifies bearer tokens and loads the user; the admin middleware checks the user role.

Figure 1: FreshKart system architecture

React Frontend ---> Express API ---> MongoDB Database

3.3 Frontend Areas

The route map includes public landing, login, and registration pages; protected shop, product, cart, checkout, profile, order, and subscription screens; and a nested administration area for dashboard, products, orders, and users. Cart and authentication state are provided through React contexts.

4. Data Source and Database Design

4.1 Data Source

The repository includes JavaScript seed data for users and products and a seeder that clears and repopulates orders, subscriptions, products, and users. The seeder is intended for initialization or reset, so it must be treated carefully in an environment containing valuable data. The repository does not provide a captured production dataset or verified record counts for this report.

4.2 Collections and Main Fields

Table 3: FreshKart MongoDB document model

MODEL / COLLECTION	KEY CONTENT	RELATIONSHIP / DESIGN
User / users	Name, unique email, password hash, admin flag, addresses, legacy address and phone, timestamps.	Addresses are embedded; user is referenced by orders and subscriptions.
Product / products	Name, brand, image paths, category, description, reviews, rating, prices, stock, active/image flags.	Reviews are embedded subdocuments; admin/user and reviews refer to users.
Order / orders	User, item snapshots, shipping address, payment values, totals, status, tracking history.	Order items and tracking events are embedded; user/product/subscription are referenced.
Subscription / subscriptions	User, product, quantity, frequency, dates, billing type, next delivery, status.	References the user and product; frequency/status use enumerated values.

4.3 Embedding and Referencing

The model embeds values that are commonly read with their parent: addresses within users, reviews within products, order lines within orders, and tracking events within orders. This keeps common page reads compact and makes historical order lines retain item names, prices, and image values. References connect orders and plans to their owning user and product, supporting lookup and population when details are needed.

4.4 Validation and Indexes

Mongoose schemas declare required fields, defaults, enums, timestamps, and a unique email field. The models do not declare explicit compound indexes or MongoDB collection-level JSON Schema validators in the reviewed files. MongoDB creates an index for the unique email constraint when the model index is built; query-specific indexes for order history, subscription due dates, product browsing, and administrative filtering should be considered and verified against actual query plans.

5. Application Implementation

5.1 Authentication and User Management

Registration checks for an existing email and creates a user. A pre-save hook hashes modified passwords with bcryptjs. Login verifies the password and returns a JWT plus basic profile fields. The protection middleware verifies the token and retrieves the user without the password. The OTP controller stores temporary codes in process memory with a ten-minute expiry and removes them after successful verification.

5.2 Product Catalog

Products include descriptive, pricing, inventory, image, and availability fields. Administrators can create and update products and upload or remove images. Signed-in customers can submit one review per product

according to controller logic; the product document stores the review subdocument and recalculated average/count.

5.3 Checkout and Orders

The order controller checks every requested product and available quantity, then reduces stock, creates an order, and optionally creates subscription records for order lines marked recurring. Orders store item snapshots, shipping data, payment method, price breakdown, status, and an initial tracking event. Admin endpoints update the status or mark an order delivered; COD orders are marked paid on delivery.

5.4 Subscription Lifecycle

Plan creation stores product, quantity, frequency, start date, next delivery date, optional end date/duration, billing type, and active status. Customers can list their plans and update status, quantity, or frequency subject to ownership checks. Cancellation also attempts to cancel linked orders that have not progressed beyond the packed stage.

5.5 Administration

The frontend defines admin-only route components and the backend uses protect/admin middleware on most administrative routes. The administration suite includes product, order, user, and dashboard screens. Advanced analytics, offers, and settings are represented by placeholder pages in the route map.

6. MongoDB Operations and Integration

6.1 Mongoose Integration

The database configuration calls mongoose.connect using the MONGO_URI environment variable. The server loads dotenv, starts the database connection, configures CORS and JSON parsing, serves uploaded files, mounts API routes, and installs not-found and general error handlers. Model methods and hooks provide password comparison and hashing behavior.

6.2 CRUD Mapping

Table 4: CRUD operations in the application

OPERATION	EXAMPLES IN FRESHKART
Create	Register user, add product, create order, add subscription, submit review.
Read	Fetch catalog/product, read profile, list customer orders/plans, load admin records.
Update	Edit profile/product, adjust plan quantity/frequency/status, update order lifecycle.
Delete	Remove a product or user, delete uploaded product image; seed reset deletes domain records.

6.3 Document Design Considerations

Embedding order lines and tracking events favors retrieving a complete order in one document read. Product reviews are also embedded, which is convenient at modest scale but may grow without bound for highly reviewed products. The implementation benefits from MongoDB’s flexible document structure, while correctness still depends on application-level validation and transaction design.

6.4 Queries and Aggregation

The reviewed API uses standard Mongoose model operations such as find, findById, countDocuments, updateMany, and save. The analytics endpoint counts orders, products, and users and sums paid order totals in

application memory. No aggregation pipeline implementation was found in the reviewed controller code; database-side aggregation could reduce transfer and memory use for larger order histories.

7. Results and Implementation Analysis

7.1 Implemented Capabilities

Static inspection confirms a broad set of application paths and data models: authentication, catalog browsing and administration, checkout, order history and tracking, product reviews, and recurring plan management. The repository README also describes the intended customer flow and local setup. These findings establish implementation presence in source, not successful end-to-end behavior in a running environment.

7.2 Important Findings

Table 5: Important implementation findings

PRIORITY	FINDING	IMPACT
High	POST /api/orders/generate is mounted without protect/admin middleware, despite its controller comment describing admin access.	Unauthenticated callers may trigger scheduled order generation.
High	Generated orders set only shippingAddress.address, while the Order schema requires address, city, and postalCode.	Generated documents may fail validation and prevent recurring fulfillment.
High	Checkout validates stock and then saves decrements in separate reads/writes without a transaction or atomic conditional update.	Concurrent checkouts or partial failures can cause inconsistent inventory and order state.
Medium	Subscription generation filters active plans by nextDeliveryDate but does not check endDate or duration in the shown loop.	A plan may continue generating after its intended end date.
Medium	A recurring order can create more than one subscription when multiple order lines are marked recurring, but a single order-level subscriptionId is overwritten.	Order-to-plan linkage can be ambiguous for multi-plan orders.
Medium	The due-date endpoint is a manually callable route; no scheduler or background worker configuration is present in the reviewed files.	Recurring orders require an external trigger to run reliably.
Medium	Analytics loads all paid orders and sums totals in application memory.	Memory and response time grow with paid-order volume.

7.3 Verification Status

This report does not include live database output, automated test results, benchmark measurements, or deployment evidence. The repository’s backend package contains no meaningful test script. Findings above are derived from schema and controller behavior and should be verified with integration tests before release.

8. Discussion and Recommendations

8.1 Strengths

- The application has a coherent separation between frontend pages, API routes, controllers, middleware, and data models.
- MongoDB document modeling aligns with order snapshots, addresses, reviews, and delivery history.
- Password hashing, JWT protection, role checks, and ownership checks are present in core flows.

- The subscription model captures useful schedule and billing choices and is integrated with checkout.
- The interface includes customer and administrator workflows instead of a catalog-only demonstration.

8.2 Priority Improvements

Table 6: Priority improvements

ORDER	RECOMMENDATION
1	Protect the order-generation route with authentication and admin authorization; use a scheduler or queue with idempotency and audit logs.
2	Build generated shipping addresses from validated user address records and ensure all required Order fields and totals are populated.
3	Use a MongoDB transaction or atomic stock reservation pattern so inventory changes and order creation succeed or fail together.
4	Enforce subscription end dates, pause/cancel state, retry rules, and next-date behavior in a dedicated scheduling service.
5	Define explicit indexes based on observed access patterns, including product discovery, customer order history, and due subscriptions.
6	Add input validation, rate limits for OTP, secure OTP storage, and tests for authorization, checkout concurrency, and plan lifecycle.
7	Move paid-order totals to a database aggregation and add pagination to catalog, user, and order list endpoints.

8.3 Security and Operations

OTP data is held in an in-memory object, which is lost on process restart and is not shared across multiple server instances. Verification should use a cryptographically secure generator, store only a protected representation with expiry, enforce attempt limits, and deliver codes through a configured provider. Deployment should also constrain CORS, validate environment configuration, secure upload handling, and avoid returning password fields in all administrative responses.

9. Conclusion and Future Scope

9.1 Conclusion

FreshKart presents a substantial full-stack foundation for a subscription-oriented grocery storefront. Its React application connects to a Node.js and Express API that uses Mongoose models for users, products, orders, and recurring plans. The source demonstrates core ecommerce workflows and MongoDB document modeling choices appropriate to the prototype's scope. Before dependable production use, the scheduled-order endpoint, generated order validation, inventory consistency, subscription expiry, and recurring execution need focused engineering and integration verification.

9.2 Future Scope

- Integrate a payment gateway and payment reconciliation workflow.
- Run recurring fulfillment through a secured, retryable background scheduler or queue.
- Add delivery partner integration and customer notifications through email/SMS.
- Add address validation, delivery-slot selection, and serviceability checks.
- Expand admin analytics with database aggregation and exportable reports.
- Add automated tests, CI checks, structured logs, monitoring, and deployment documentation.
- Consider a separate reviews collection or bounded review strategy if product review volume grows.

References

[1] FreshKart project repository and README, supplied with this report, accessed 29 October 2026.

[2] MongoDB Inc., MongoDB Manual, https://www.mongodb.com/docs/manual/.

[3] Mongoose, Mongoose Documentation, https://mongoosejs.com/docs/.

[4] Express.js, Express Documentation, https://expressjs.com/.

[5] React, React Documentation, https://react.dev/.

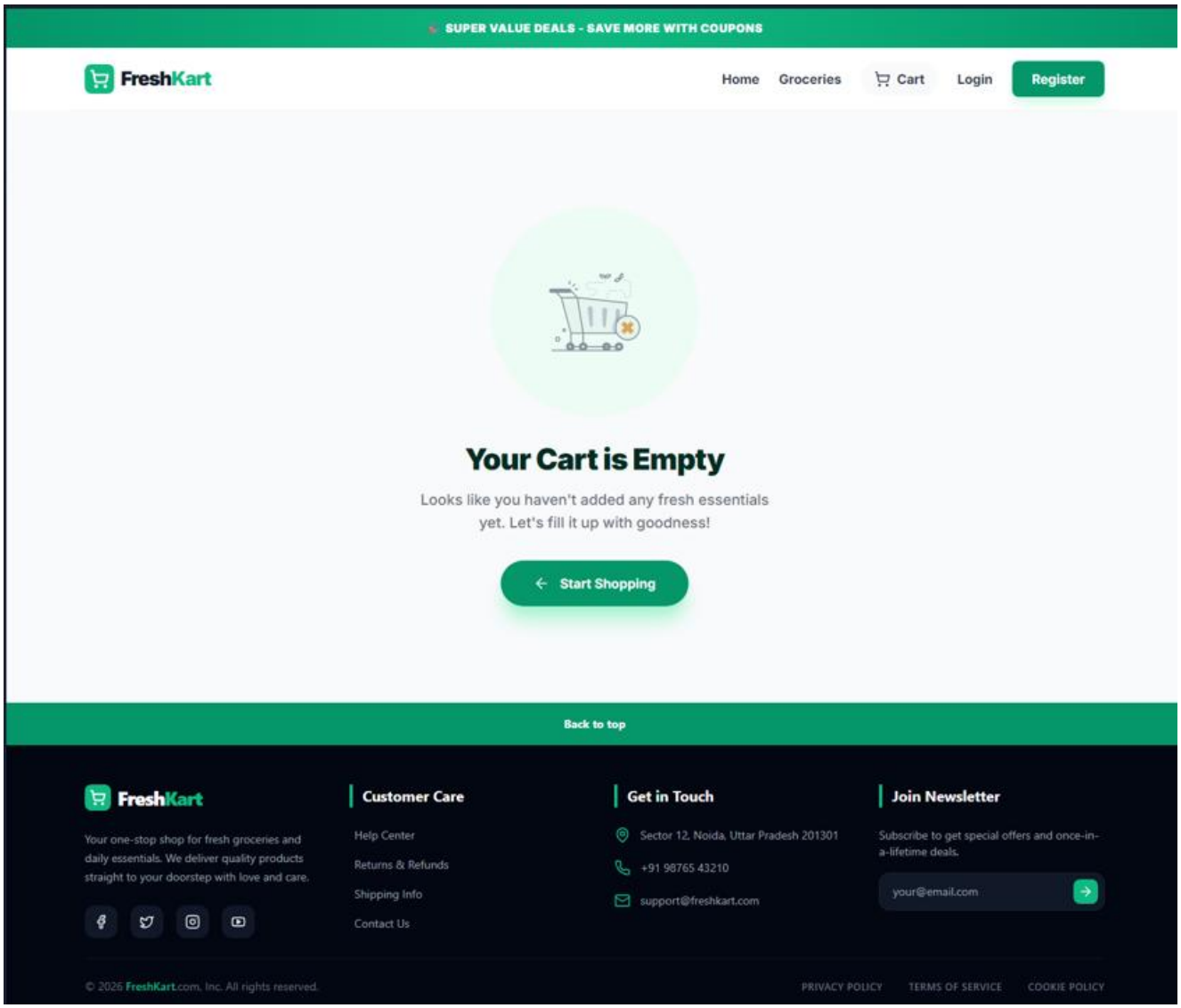
[6] OpenJS Foundation, Node.js Documentation, https://nodejs.org/docs/.

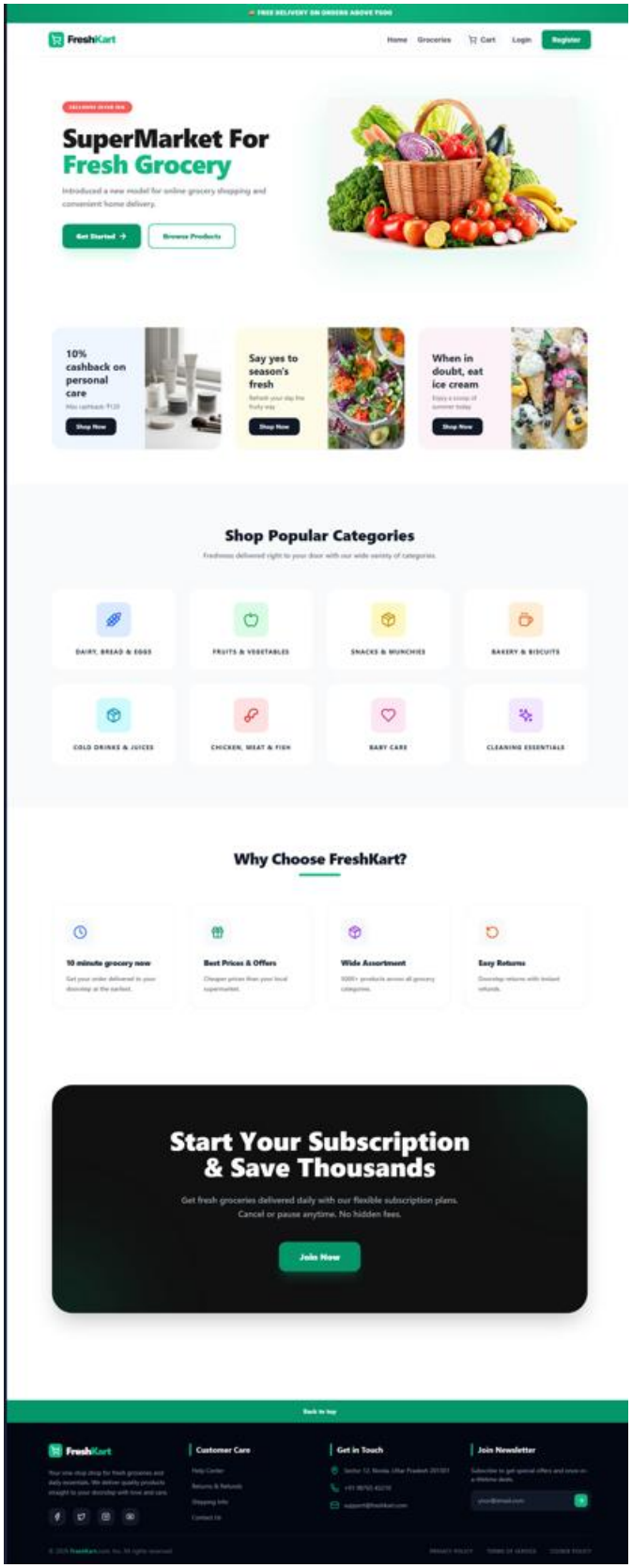
[7] Auth0, JSON Web Token Introduction, https://jwt.io/introduction.

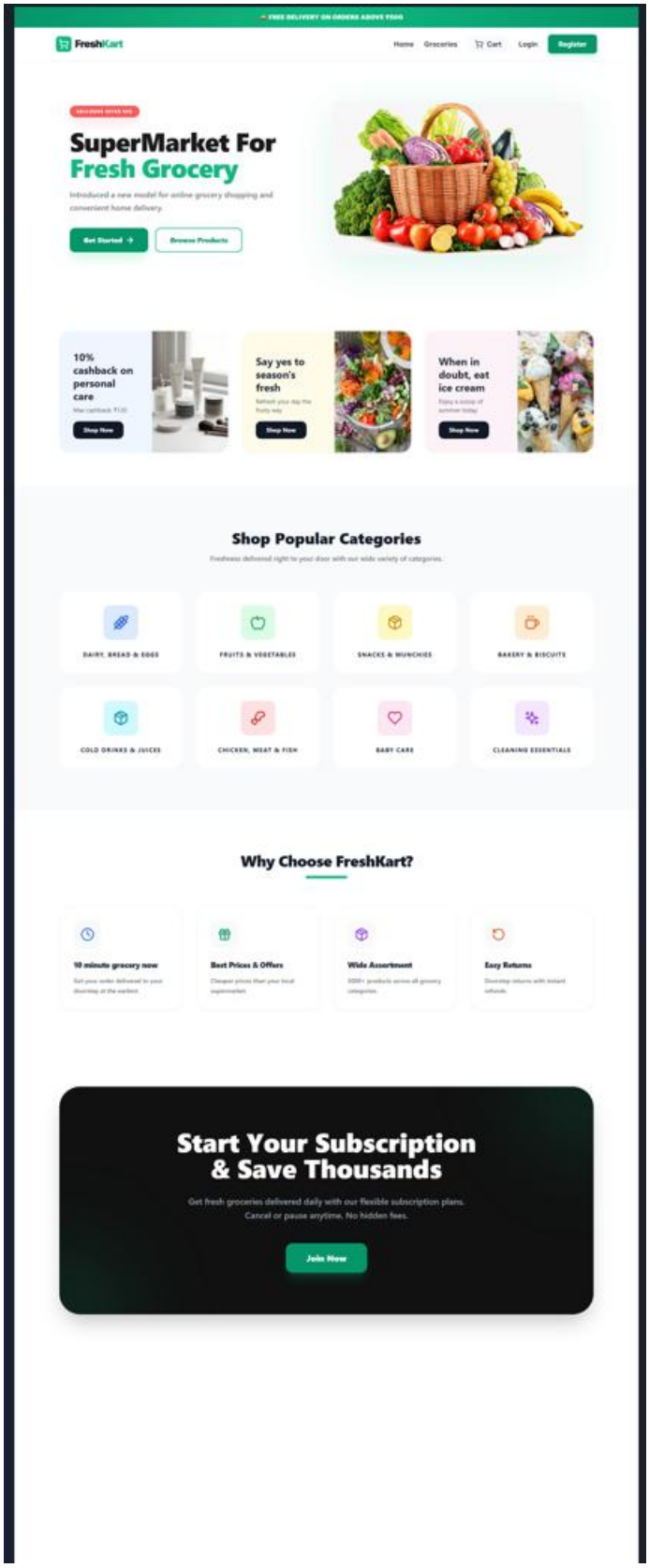
GitHub Link

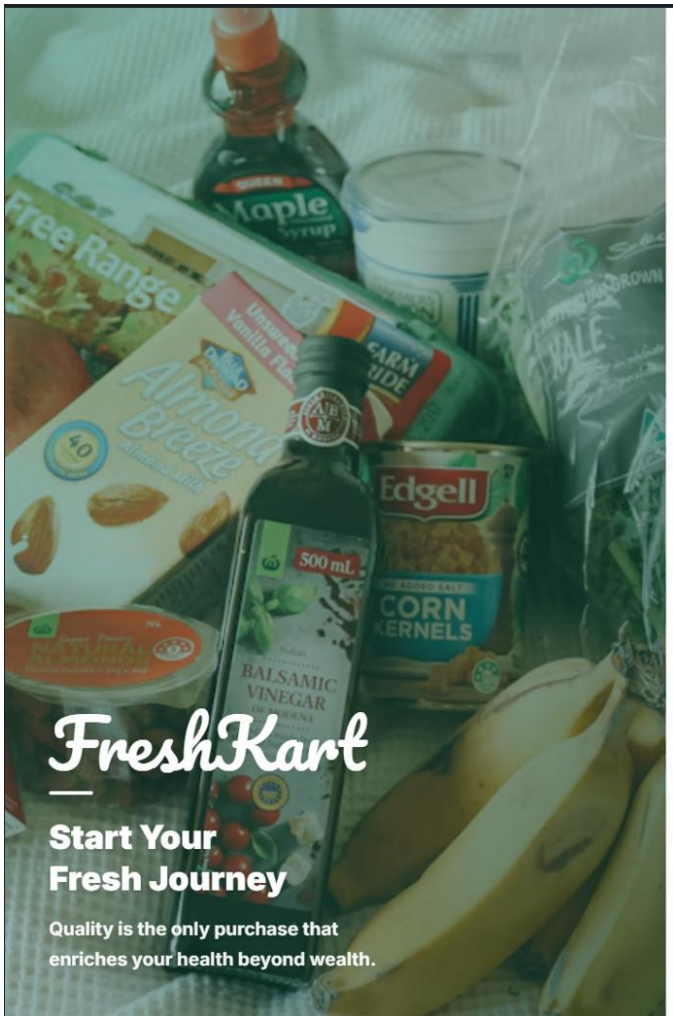
https://github.com/AaruhiShrivastva/FreshKart-Subscription-Based-Ecommerce-Platform

Website Preview:




FreshKart
Home
Groceries
Cart
Login
Register
SuperMarket For
Fresh Grocery
Introduced a new model for online grocery shopping and convenient home delivery.
Get Started →
Browse Products
10% cashback on personal care
Shop Now
Say yes to season's fresh
Shop Now
When in doubt, eat ice cream
Shop Now
Shop Popular Categories
Freshness delivered right to your door with our wide variety of categories.
DAIRY, BREAD & EGGS
FRUITS & VEGETABLES
SNACKS & MUNCHIES
BAKERY & BISCUITS
COLD DRINKS & JUICES
CHICKEN, MEAT & FISH
BABY CARE
CLEANING ESSENTIALS
Why Choose FreshKart?
10 minute grocery now
Best Prices & Offers
Wide Assortment
Easy Returns
Start Your Subscription & Save Thousands
Get fresh groceries delivered daily with our flexible subscription plans. Cancel or pause anytime. No hidden fees.
Join Now
Back to top
FreshKart
Customer Care
Get in Touch
Join Newsletter


FREE DELIVERY ON ORDERS ABOVE ₹500
FreshKart
Home
Groceries
Cart
Login
Register
SuperMarket For
Fresh Grocery
Introduced a new model for online grocery shopping and convenient home delivery.
Get Started →
Browse Products
10% cashback on personal care
Shop Now
Say yes to season's fresh
Shop Now
When in doubt, eat ice cream
Shop Now
Shop Popular Categories
Freshness delivered right to your door with our wide variety of categories.
DAIRY, BREAD & EGGS
FRUITS & VEGETABLES
SNACKS & MUNCHIES
BAKERY & BISCUITS
COLD DRINKS & JUICES
CHICKEN, MEAT & FISH
BABY CARE
CLEANING ESSENTIALS
Why Choose FreshKart?
10 minute grocery now
Best Prices & Offers
Wide Assortment
Easy Returns
Start Your Subscription & Save Thousands
Get fresh groceries delivered daily with our flexible subscription plans.
Cancel or pause anytime. No hidden fees.
Join Now



Connect

ESTABLISH YOUR PROFILE

FULL NAME
Johnathan Doe

EMAIL ADDRESS
john@freshkart.com

PASSWORD
••••••••

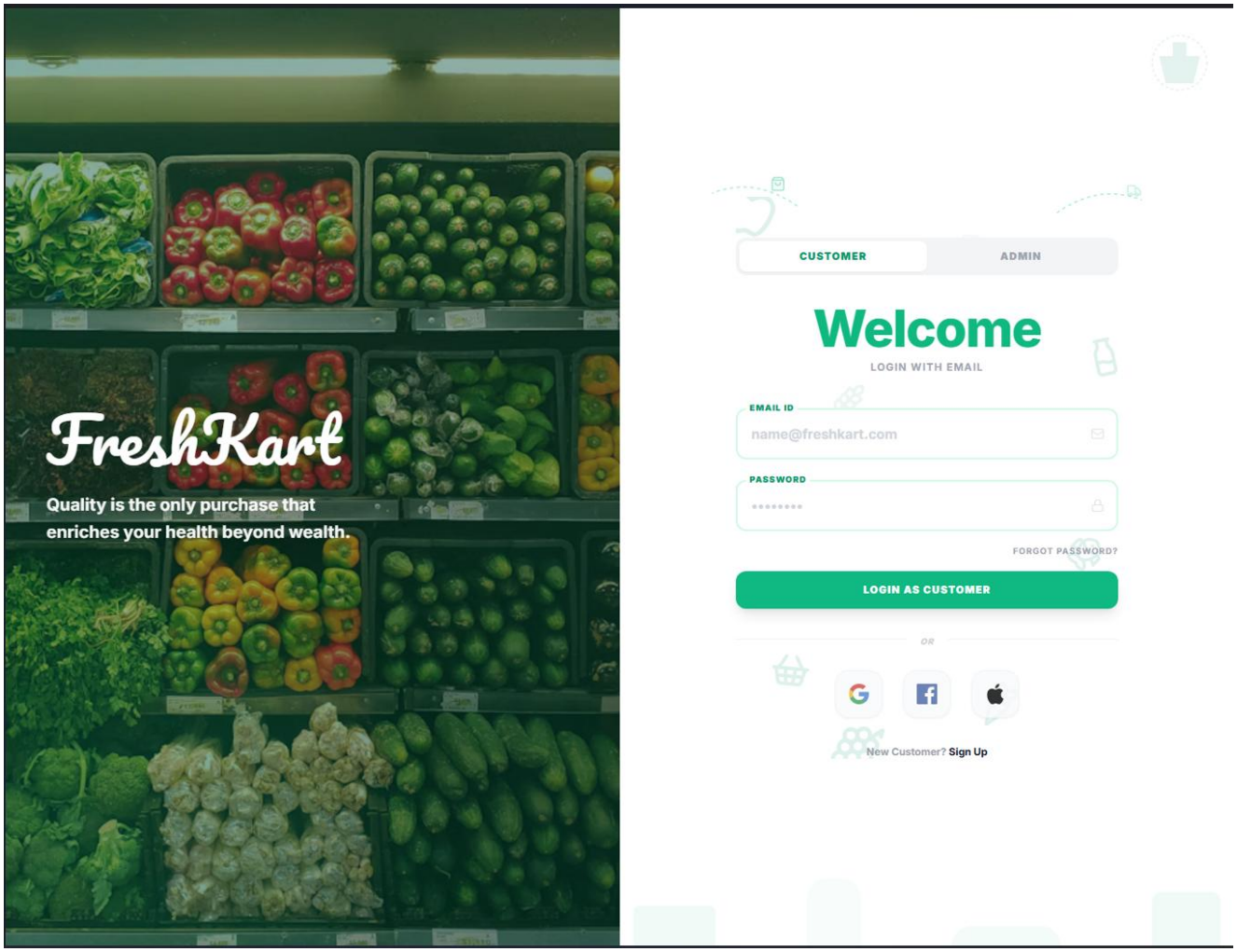
CONFIRM
••••••••

PHONE NUMBER
10-digit mobile number

AUTHENTICATE & VERIFY

OR JOIN WITH

Account exists? Sign In


FreshKart
Quality is the only purchase that enriches your health beyond wealth.
CUSTOMER
ADMIN
Welcome
LOGIN WITH EMAIL
EMAIL ID
name@freshkart.com
PASSWORD
FORGOT PASSWORD?
LOGIN AS CUSTOMER
OR
New Customer? Sign Up